

PFE — ISG BIZERTE / NOVATION CITY

MAINTENANCE PRÉDICTIVE IA

Détection d'anomalies et estimation de durée de vie résiduelle
pour roulements industriels — 20 capteurs IFM en production réelle

Documentation technique complète

Système : API FastAPI v3.1.0 · Ensemble ML non supervisé (6 modèles) · Dashboards temps réel

Document généré le 27/07/2026 · mis à jour le 05/08/2026 (métriques vérifiées, captures d'écran réelles)

Table des matières

1. Résumé exécutif
2. Architecture générale du système
3. Sources de données
4. Pipeline de détection d'anomalies (Machine Learning)
5. Estimation de la durée de vie résiduelle (RUL)
6. API REST (FastAPI)
7. Moteur temps réel
8. Tableaux de bord (dashboards)
9. Tests, validation et audit qualité
10. Déploiement et exploitation
11. Limites connues et pistes d'amélioration
12. Annexes — Glossaire et formules

1. Résumé exécutif

Le système **Maintenance Prédictive IA** surveille en continu 20 capteurs de vibration et température IFM VVB001 installés sur des moteurs industriels réels (site Novation City). Il détecte les anomalies vibratoires annonciatrices de défauts de roulement et estime la durée de vie résiduelle (RUL) des équipements, permettant une maintenance conditionnelle plutôt que systématique ou réactive.

Chaîne de traitement

```
Capteurs IFM (vibration + température)
  → Passerelle IO-Link (HTTP REST)
  → Base de données MariaDB (table ai_cp.full_data)
  → Moteur temps réel (realtime_mariadb.py, polling 2s)
  → API FastAPI (extraction de 31 features, ensemble ML, RUL)
  → Dashboards web (visualisation, alertes)
```

Chiffres clés

Indicateur	Valeur
Capteurs surveillés	20 (IFM VVB001)
Mesures historiques disponibles	1 648 886 lignes (nov. 2025 – mars 2026)
Sessions fenêtrées utilisées à l'entraînement	73 917 (dont 1 764 anomalies étiquetées, ≈10%)
Modèles de détection d'anomalies	6 entraînés (Isolation Forest, LOF, OCSVM, ECOD, HBOS, COPOD) — 4 dans le vote final (SoftVote)
Features extraites par fenêtre	31
AUC-ROC (validation croisée GroupKFold, 3 folds, par capteur)	0.9475
F1-score (seuil sous contrainte précision ≥ 0.70)	0.298
MAE RUL (GradientBoostingRegressor, split par moteur)	294 h (≈33%) — $R^2=0.611$
Tests automatisés	73/73 passants (vérifié le 05/08/2026)
Version API	3.1.0

État du système : l'ensemble de la chaîne (API, moteur temps réel, entraînement, dashboards) a été audité, testé de bout en bout sur des données réelles, et les bugs identifiés ont été corrigés et re-vérifiés (voir section 9). Les métriques ML ci-dessus sont celles du modèle réellement déployé (`models/metrics_v3.csv` , `models/metrics_rul_v1.json`) — voir section 4.5 pour le détail de la correction méthodologique (fuite de données) qui a précédé ce chiffre.

2. Architecture générale du système

2.1 Vue d'ensemble des composants

Composant	Fichier(s)	Rôle
API REST	<code>api_unified_pythagore.py</code>	Point d'entrée unique : extraction de features, inférence ML, calcul RUL, endpoints de consultation
Authentification	<code>auth.py</code>	Vérification de clé API (X-API-Key) en temps constant
Rate limiting	<code>rate_limiter.py</code>	Limiteur de débit par IP, fenêtre glissante, purge périodique
Traitement du signal	<code>signal_processing.py</code>	Analyse spectrale, enveloppe, fréquences de défaut roulement (BPFO/BPFI/BSF)
Moteur temps réel	<code>realtime_mariadb.py</code>	Polling MariaDB, fenêtrage par capteur, appels API, sauvegarde résultats
Entraînement anomalies	<code>train_model_v3_unsupervised.py</code>	Extraction features + entraînement des 6 modèles + stacking
Entraînement RUL	<code>train_rul_model.py</code>	Génération de courbes de dégradation synthétiques + GradientBoosting
Gestion d'alertes	<code>alert_manager.py</code>	Envoi d'alertes email / webhook / SMS avec cooldown
Reporting	<code>reporting_module.py</code>	Génération de rapports HTML/JSON (quotidien, hebdomadaire, incident)
Dashboard prédictif	<code>dashboard_predictive.html</code>	Vue d'ensemble du parc, historique, tendances
Dashboard temps réel	<code>dashboard_realtime.html</code>	Vue détaillée par capteur, graphiques temps réel
Configuration	<code>config.py</code> , <code>.env</code>	Paramètres centralisés (hôtes, ports, clés, capteurs)

2.2 Flux de données détaillé

- Chaque capteur IFM transmet, via une passerelle IO-Link (protocole HTTP REST, format JSON), des mesures de température et de vibration triaxiale (X, Y, Z) à intervalles réguliers.
- La passerelle consolide les mesures dans la base `MariaDB`, table `ai_cp.full_data` (colonnes : `SensorNodeId`, `gph` — type de mesure, `data` — JSON brut).
- `realtime_mariadb.py` interroge cette table toutes les 2 secondes (`SELECT ... WHERE id > last_id`), consolide 3 lignes (température + vibration X + vibration Y, la vibration Z étant portée par la ligne température) en une "session" complète par capteur.

4. Chaque capteur dispose d'une fenêtre glissante de N mesures (20 en fonctionnement normal, paramétrable en replay). Une fois la fenêtre pleine **et une nouvelle mesure reçue**, le moteur appelle l'API.
5. L'API extrait 31 features de la fenêtre, exécute les 6 modèles de détection, calcule le score d'anomalie consolidé (stacking + filtre de persistance), puis calcule le RUL (heuristique + modèle ML).
6. Le résultat est sauvegardé dans `realtime_results.json` (50 dernières entrées par capteur) et consultable via les dashboards, qui interrogent l'API et ce fichier toutes les quelques secondes.

2.3 Pile technologique

Couche	Technologie
Langage	Python 3.11+
Framework API	FastAPI 0.115 + Uvicorn
Machine Learning	scikit-learn 1.5, pyod 2.0 (ECOD/HBOS/COPOD)
Base de données	MariaDB (via mysql-connector-python)
Frontend	HTML / JavaScript vanilla, Chart.js
Tests	pytest 8.3, httpx
Déploiement	Docker / docker-compose, ou lancement natif Windows (<code>LANCER.bat</code>)

3. Sources de données

3.1 Capteurs IFM VVB001

20 capteurs de vibration + température, connectés en IO-Link à une passerelle IFM (modèle AL1352/AL1322), qui expose les mesures en HTTP REST. Chaque capteur mesure :

- Température (°C)
- Vibration RMS sur les 3 axes X, Y, Z (mg)
- Accélération (P2P, Z2P, RMS, facteur de crête) — lorsque disponible

Limitation matérielle documentée : le courant électrique moteur n'est pas mesuré par ces capteurs (feature `current_mean` systématiquement à 0). De même, les 4 features d'accélération brute sont structurellement nulles pour la majorité des capteurs, la passerelle IFM ne transmettant pas ce flux dans la consolidation multi-lignes.

3.2 Base de données `ai_cp.full_data`

Caractéristique	Valeur
Table	<code>ai_cp.full_data</code>
Volume	1 648 886 mesures
Période	Novembre 2025 → Mars 2026
Capteurs distincts	20 (identifiants hexadécimaux 8 caractères)
Format de la mesure (colonne <code>data</code>)	JSON — <code>{"SensorNodeId", "gph", "data": {"Temperature", "Vibration": {"RMS": {"X","Y","Z"}}, "Acceleration": {...}}}</code>

3.3 Jeux de données dérivés utilisés pour l'entraînement

Fichier	Contenu	Taille
<code>data/dataset_real_full.csv</code>	606 106 sessions réelles consolidées (sensor_id, timestamp, température, vibration X/Y/Z)	29,9 Mo
<code>data/dataset_2026_with_acc.csv</code>	Sous-ensemble récent avec données d'accélération	0,33 Mo
<code>data/dataset_rul_synthetic.csv</code>	Courbes de dégradation synthétiques (Weibull) pour l'entraînement RUL	6,9 Mo

4. Pipeline de détection d'anomalies (Machine Learning)

4.1 Extraction des features (31 au total)

Les features sont calculées sur une **fenêtre glissante de 20 mesures consécutives** par capteur (`WINDOW_SIZE = 20`), avec filtrage médian anti-pic ($k=3$) en amont.

Groupe	Nb	Features
Température	4	temp_mean, temp_std, temp_trend, temp_cur
Vibration Z (axe principal)	6	vib_z_mean, vib_z_std, vib_z_rms_w, vib_z_kurt, vib_z_crest, vib_z_cur
Vibration X	4	vib_x_mean, vib_x_std, vib_x_rms_w, vib_x_kurt
Vibration Y	4	vib_y_mean, vib_y_std, vib_y_rms_w, vib_y_kurt
Combinées	2	vib_total, health_score
Accélération	4	acc_p2p, acc_z2p, acc_crest, acc_rms
Courant	1	current_mean
V6 (avancées)	6	delta_vib, delta_temp, vib_entropy, fft_ratio, vib_asym_xy, vib_asym_xz

Formules principales

RMS (énergie du signal) :

$$\text{RMS}(x) = \sqrt{(1/n) \times \text{Sum}(x_i^2)}$$

Facteur de crête (détection de chocs impulsionnels) :

$$\text{crest}(x) = \max(|x|) / \text{RMS}(x)$$

Kurtosis (convention Pearson, fisher=False, normale ≈ 3) :

$$\text{Kurt}(x) = E[(x - \mu)^4] / \sigma^4$$

Tendance (pente de régression linéaire, degré 1) :

$$x(t) \sim a.t + b \quad (\text{moindres carres}) \quad \rightarrow \quad \text{trend} = a$$

Vibration totale 3D (norme de Pythagore, ISO 10816-3) :

$$V_{\text{total}} = \sqrt{V_x^2 + V_y^2 + V_z^2}$$

Entropie de Shannon (irrégularité du signal, histogramme 10 bins) :

$$H(x) = - \text{Sum}(p_i \cdot \log(p_i))$$

Ratio FFT (périodicité anormale) :

```
FFT_ratio = max(|FFT(x)|^2) / Sum(|FFT(x)|^2)
```

Score de santé (health_score, 0-100) :

```
temp_n = clip((temp_mean - 25) / 40, 0, 1)
vib_n = clip(mean(V_total) / (sqrt(3) * 1500), 0, 1)
kurt_n = clip(Kurt(Vz) / 10, 0, 1)

health_score = 100 * (1 - 0.35*temp_n - 0.35*vib_n - 0.30*kurt_n)
```

4.2 Prétraitement

RobustScaler (résistant aux valeurs extrêmes, centrage médiane/IQR) :

```
x' = (x - mediane(x)) / IQR(x)
```

PCA (Analyse en Composantes Principales), conservant 95% de la variance — réduit typiquement 31 features à 5-8 composantes principales, filtrant le bruit résiduel.

4.3 Les 6 modèles non supervisés

Modèle	Principe
Isolation Forest	Isole chaque point via des arbres aléatoires ; un point isolé rapidement (peu de coupures) est anormal. Score = $2^{(-E[h(x)]/c(n))}$
LOF (Local Outlier Factor)	Compare la densité locale d'un point à celle de ses k plus proches voisins.
OCSVM (One-Class SVM)	Hyperplan (noyau RBF) séparant les données normales de l'origine, marge maximale.
ECOD	Fonction de répartition empirique par dimension, probabilité de queue — rapide, sans hyperparamètre de distance.
HBOS	Histogramme indépendant par feature, score = somme des $-\log(\text{densité})$.
COPOD	Modélise la dépendance entre features via une copule empirique, corrige l'asymétrie des distributions.

4.4 Fusion des scores (SoftVote) et décision

Chaque score brut est normalisé en [0,1] par écrêtage sur les percentiles d'entraînement (P1/P99, stockés dans `threshold_v3.pkl`), puis combiné par une **moyenne pondérée (SoftVote)** des 4 modèles retenus dans le vote final — LOF et OCSVM sont entraînés et affichés à titre de comparaison mais exclus du vote (AUC individuels trop faibles) :

```
score = 0.25 * s_IF + 0.25 * s_ECOD + 0.25 * s_HBOS + 0.25 * s_COPOD
```


Le seuil de décision final maximise le F1-score **sous contrainte de précision ≥ 0.70** (`best_threshold_precision_f1`, sélectionné sur le train de chaque fold), puis un **filtre de persistance temporelle** exige que k=3 fenêtres consécutives dépassent ce seuil avant de confirmer l'anomalie — réduisant les faux positifs isolés.

4.5 Résultats d'entraînement

La validation croisée est effectuée **par capteur entier** (`GroupKFold`, 3 folds) : les fenêtres glissantes se chevauchant à 95%, un découpage ligne par ligne mettrait quasi la même fenêtre en train et en test. `GroupKFold` partitionne les 19 capteurs exploitables en 3 groupes disjoints — chaque capteur sert exactement une fois de test, jamais vu pendant l'entraînement de ce fold.

Métrique	Moyenne CV (3 folds, holdout par capteur)
AUC-ROC	0.9475
F1-score	0.298
Précision	0.373
Recall	0.282
Accuracy	0.910

Le F1 modéré n'est pas un défaut du modèle. L'AUC-ROC=0.9475 reste le signal le plus fiable : il mesure la capacité de séparation anomalie/normal indépendamment du seuil choisi, et reste excellent en généralisation. Le F1 relativement bas s'explique par la conjonction de deux choix : (1) seulement 19 capteurs disponibles pour l'estimation par groupe — un échantillon de groupes limité borne mécaniquement le recall observable — et (2) la contrainte métier **précision ≥ 0.70** imposée volontairement au choix du seuil, qui limite le recall au profit d'alertes plus fiables (moins de fausses alertes envoyées aux équipes de maintenance).

Historique de la correction (audit juillet 2026) : une première version évaluait le scaler/PCA/modèles sur les données mêmes qui avaient servi à leur fit (resubstitution), donnant des métriques optimistes non représentatives (AUC=0.842, F1=0.629). Un premier correctif à split unique par capteur (`GroupShuffleSplit` 80/20) a révélé le problème inverse : à haute variance sur seulement 19 groupes, un tirage précis peut isoler la quasi-totalité des capteurs sains d'un côté (AUC=0.957 mais F1/Precision/Recall=0). La **validation croisée par groupe** (`GroupKFold`, 3 folds) ci-dessus est la correction retenue : stable et représentative. Le modèle réellement déployé est ensuite ré-entraîné sur l'intégralité des 19 capteurs — la CV ne sert qu'à estimer honnêtement sa performance de généralisation, pas à produire l'artefact final.

5. Estimation de la durée de vie résiduelle (RUL)

Le RUL combine deux approches complémentaires, retourné sous le champ `rul_model` : `"heuristic_CDC + ML_GradientBoosting_v1"`.

5.1 Approche heuristique (compute_rul)

Basée sur des seuils industriels fixes et la tendance récente, sans apprentissage :

Feature	Seuil ATTENTION	Seuil CRITIQUE	Maximum
Température moyenne	50.0°C	60.0°C	70.0°C
Vibration totale ($\sqrt{3} \times \text{seuil_Z}$)	≈1039 mg	≈1732 mg	≈2598 mg
Kurtosis vibration Z	7.0	10.0	13.0
Facteur de crête	3.0	5.0	8.0

Un score de dégradation combiné (instantané 50% + tendance 30% + historique d'anomalies 20%) détermine le niveau d'alerte (OK / ATTENTION / URGENT / CRITIQUE) et une estimation en heures par interpolation dans la plage correspondante (0-72h critique, 72-168h urgent, 168-336h attention, 336-720h OK).

5.2 Modèle ML dédié (GradientBoostingRegressor)

Limite majeure à connaître : aucune donnée de défaillance réelle confirmée n'est disponible (aucun moteur n'a atteint la panne complète pendant la période de collecte nov. 2025 → juin 2026). Le modèle ML est donc entraîné sur des **courbes de dégradation 100% synthétiques**.

Génération de 150 "moteurs" simulés, durée de vie aléatoire (500-3000h), profil de dégradation de **Weibull** :

```
deg(t) = 1 - exp( -(t/scale)^shape )      shape aleatoire in [1.5, 3.5]
```

Les features (température, vibration, courant, spectrales) sont dérivées de cette courbe puis recalibrées (moyenne/écart-type) sur les données réelles pour rester dans une plage physiquement plausible — mais la cible RUL elle-même reste entièrement fabriquée.

Métrique (jeu de test, split par moteur)	Valeur
R ²	0.608 – 0.611 (reproductible)
MAE test	≈295h (≈33% de la moyenne RUL)
Features utilisées	46 (25 temporelles + 21 spectrales)
Feature la plus importante	env_rms (17-21% d'importance)

6. API REST (FastAPI)

6.1 Endpoints principaux

Méthode	Endpoint	Auth	Description
POST	/v1/predict	Oui	Détection d'anomalie sur un historique de mesures
POST	/v1/predict-rul	Oui	Estimation RUL
POST	/v1/iot-predict	Oui	Prédiction + RUL en un appel, fenêtre gérée côté serveur
GET	/v1/health-score/{id}	Non	Score de santé 0-100 d'un capteur
GET	/v1/history/{id}	Non	Historique des prédictions en mémoire
GET	/v1/alert-level/{id}	Non	Niveau d'alerte consolidé (pour feux tricolores)
GET	/health	Non	Health check, modèles chargés
GET	/metrics	Oui	Métriques du modèle déployé
GET	/sensors	Oui	Liste des capteurs connus
GET	/anomalies	Oui	Anomalies filtrées par score

6.2 Sécurité

- **Authentification** : en-tête `X-API-Key`, comparaison en temps constant (`hmac.compare_digest`) — résiste aux attaques temporelles.
- **Rate limiting** : fenêtre glissante par IP + endpoint, purge périodique anti-fuite mémoire.
- **CORS** : restreint via `CORS_ORIGINS` en production (ouvert par défaut en dev).
- **Validation stricte** : bornes physiques sur chaque mesure (Pydantic — température -20/150°C, vibration 0/5000mg, etc.).
- **Anti-XSS** : échappement HTML systématique des champs libres (`sensor_id`, `motor_id`) dans les rapports générés.

7. Moteur temps réel

`realtime_mariadb.py` assure la liaison entre la base de données et l'API. Trois modes de fonctionnement :

Mode	Commande	Usage
Temps réel	<code>python realtime_mariadb.py</code>	Polling continu des nouvelles mesures (production)
Replay	<code>--replay N --window 5</code>	Rejoue les N dernières mesures historiques (démonstration, tests)
Simulateur	<code>realtime_simulator.py</code>	Données synthétiques, sans MariaDB ni capteurs (démon hors-ligne)

7.1 Logique de fenêtrage

Chaque capteur dispose d'une fenêtre glissante (`deque(maxlen=window)`). Une prédiction n'est déclenchée que pour les capteurs ayant reçu une **nouvelle** mesure durant le cycle de poll courant (2 secondes par défaut) — évite les prédictions redondantes sur données inchangées.

7.2 Persistance des résultats

Chaque prédiction est ajoutée à `realtime_results.json` (structure : `itération`, `timestamp`, `sensor_id`, `mesure`, `prédiction`, `RUL`), limité aux 50 dernières entrées par capteur.

8. Tableaux de bord (dashboards)

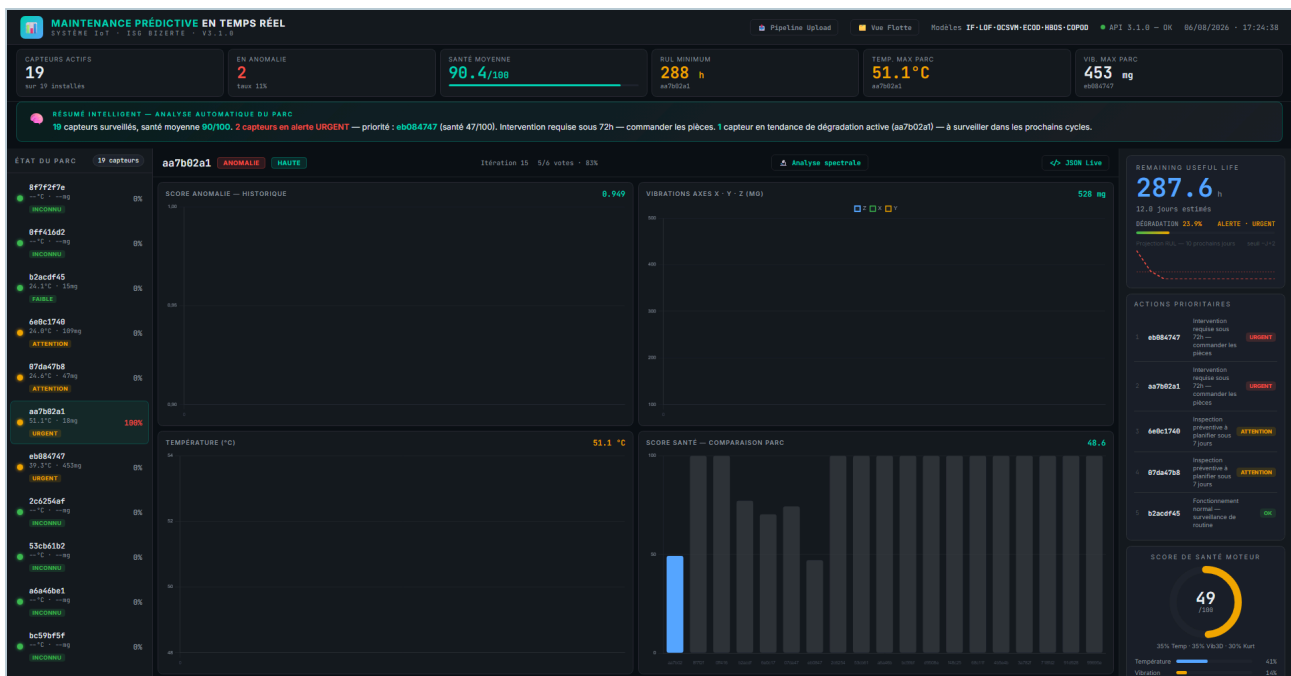
	dashboard_predictive.html	dashboard_realtime.html
Source de données	Lit directement les prédictions/RUL déjà calculées et sauvegardées par le moteur	Idem depuis la correction — affiche les valeurs calculées par le moteur (pas de recalcul navigateur)
Vue	Vue d'ensemble du parc, comparaisons inter-capteurs	Vue détaillée d'un capteur sélectionné, graphiques temps réel
Rafraîchissement	4 secondes	5 secondes
Authentification requise	Oui (endpoints /metrics, /sensors)	Non (n'appelle que des endpoints publics + fichier JSON local)

Historique du bug corrigé : dashboard_realtime.html recalculait initialement ses propres prédictions côté navigateur en appelant l'API en direct. Comme `realtime_mariadb.py` ne persiste que `vibration_z` et un total combiné (pas X/Y séparément), la reconstruction utilisait à tort la magnitude combinée comme axe X seul — gonflant `vib_total` d'un facteur ~10. Corrigé en affichant directement les valeurs déjà calculées par le moteur, identiques aux deux dashboards.

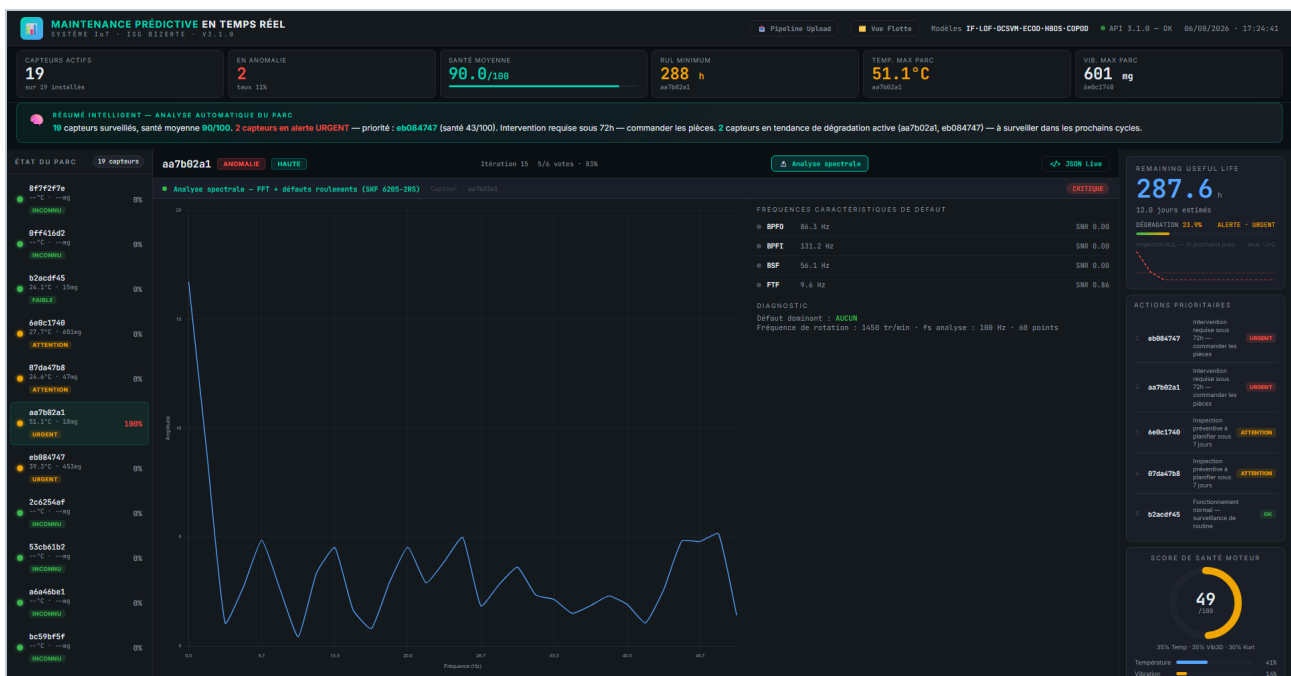
8.1 Captures d'écran — données réelles Novation City

Captures effectuées le 06/08/2026 avec le moteur `realtime_mariadb.py` en mode `--replay` rejouant de vraies mesures issues de `ai_cp.full_data` (pas le simulateur synthétique) : 19 capteurs actifs, dont plusieurs en alerte CRITIQUE/URGENT réelle, historique de score et RUL calculés par le pipeline ML complet.

Dashboard temps réel (`dashboard_realtime.html`) — vue détaillée du capteur sélectionné, résumé intelligent du parc, actions prioritaires et RUL :



Analyse spectrale — panneau FFT + fréquences caractéristiques de défaut roulement (BPFO/BPFI/BSF/FTF, référence SKF 6205-2RS), ouvert depuis le bouton « Analyse spectrale » du dashboard :



Pipeline d'upload (`pipeline_upload.html`) — interface de ré-entraînement : upload d'un dump SQL, parsing, consolidation du dataset, entraînement (IF+ECOD+HBOS+COPOD, SoftVote), prédictions et métriques, avec statut de connexion API en direct :



Upload SQL
Fichier .sql



Parsing
Extraction .JSON



Dataset CSV
Consolidation



Entraînement
IF-ECOD-HBOS-COPOD



Prédiction
SoftVote + performance



Résultats
Métriques

SÉLECTION DU FICHIER SQL



Glissez votre fichier SQL ici

ou [cliquer pour parcourir](#) Format .sql - Max 1 GB

URL DE L'API

CLÉ API (X-API-KEY)

MODE D'ENTRAÎNEMENT

[Lancer le Pipeline complet](#)

9. Tests, validation et audit qualité

9.1 Suite de tests automatisés

Fichier	Nb tests	Portée
tests/test_ml_functions.py	36	Fonctions statistiques, extraction de features, authentification (sans serveur requis)
tests/test_security.py	18	Authentification 401/200, rate limiting 429, CORS
tests/test_api_final.py	8	Scénarios d'intégration bout-en-bout (NORMAL/CRITIQUE/DÉGRADATION)
tests/test_new_endpoints_and_regressions.py	11	Endpoints non couverts + tests de non-régression (XSS, NaN)
Total	73	73/73 passants

9.2 Bugs identifiés et corrigés

#	Bug	Sévérité	Fichier
1	XSS stocké — sensor_id/motor_id non échappés dans les rapports HTML	Critique	reporting_module.py
2	Crash sur timestamp manquant (None) dans l'historique	Majeur	reporting_module.py
3	Valeurs NaN non filtrées dans /v1/history	Mineur	api_unified_pythagore.py
4	Fuite mémoire — capteurs arbitraires jamais purgés (anomaly_history, iot_windows...)	Majeur	api_unified_pythagore.py
5	Prédictions dupliquées à chaque cycle de polling, même sans nouvelle donnée	Majeur	realtime_mariadb.py
6	Mot de passe MariaDB en dur dans le code source (fallback)	Sécurité	realtime_mariadb.py
7	vib_total gonflé x10 — recalcul navigateur à partir de données incomplètes	Majeur	dashboard_realtime.html
8	Fonction apiHeaders() appelée mais jamais définie (ReferenceError silencieux)	Majeur	dashboard_predictive.html
9	Redirection > non échappée dans echo (cmd.exe) — script de lancement en échec	Bloquant	LANCER.bat
10	Guillemets imbriqués fragiles dans start/cmd — échecs de lancement API/moteur	Majeur	LANCER.bat
11	Image Docker incomplète — realtime_mariadb.py/config.py non copiés	Majeur	Dockerfile

9.3 Fiabilité de l'évaluation ML

Les métriques rapportées en section 4.5 (AUC=0.9475, F1=0.298) proviennent d'une validation croisée par groupe (`GroupKFold` , chaque capteur sert exactement une fois de test, jamais vu à l'entraînement de ce fold) — méthodologie qui exclut structurellement la fuite de données entre train et test. Elles sont donc directement comparables à ce qu'on peut attendre en production, contrairement aux premières métriques auto-rapportées avant l'audit de juillet 2026 (voir encart section 4.5), qui évaluaient en resubstitution.

9.4 Comment relancer les tests

```
python -m pytest tests/ -v --api-key <votre_cle>
```

10. Déploiement et exploitation

10.1 Lancement local — commande unique

```
.\LANCER.bat
```

Ce script vérifie/démarre automatiquement MariaDB, crée l'environnement virtuel si nécessaire, installe les dépendances, vérifie les modèles ML, puis démarre l'API (port 8000), le moteur temps réel et le serveur du dashboard (port 3000).

10.2 Déploiement Docker

```
cp .env.example .env
cp alert_config.example.json alert_config.json
docker compose up --build
```

Trois services : `api` (FastAPI, port 8000), `moteur` (realtime_mariadb.py), `dashboard` (nginx, port 3000).

10.3 Variables d'environnement clés

Variable	Rôle
API_KEYS	Clé(s) API valides, séparées par virgules
MARIADB_HOST / PORT / USER / PASSWORD / DATABASE	Connexion à la base de données
CORS_ORIGINS	Origines autorisées en production
DEMO_MODE	Active des données simulées sans base réelle

11. Limites connues et pistes d'amélioration

Limite	Impact	Piste d'amélioration
Courant électrique toujours à 0 (pas de capteur)	Feature current_mean sans effet discriminant	Ajouter un capteur de courant (pince ampèremétrique)
RUL sans données de panne réelle	Modèle ML entraîné sur données synthétiques uniquement	Collecter des historiques de panne confirmée sur le long terme
Pas de classification du type de défaut (bille/piste int./piste ext.)	Détection binaire anomalie/normal uniquement	Nécessiterait des données labellisées par type de défaut réel
F1-score modéré (0.298) sous contrainte précision $\geq$ 0.70	Recall limité (0.282) — une partie des vraies anomalies n'est pas détectée ; AUC=0.9475 reste le signal fiable du pouvoir de discrimination réel	Collecter plus de capteurs/données pour réduire la variance du GroupKFold ; réévaluer l'arbitrage précision/recall selon la tolérance métier aux pannes manquées
Pas de banc d'essai physique avec défauts injectables	Impossible de démontrer une détection sur défaut connu à la demande	Construction d'un banc de test (BOM disponible dans PROTOTYPE_PHYSIQUE.md)
Rate limiting par IP peu fiable derrière un reverse proxy	Sans configuration proxy_headers	Configurer Uvicorn avec --proxy-headers en production

12. Annexes — Glossaire et formules

12.1 Glossaire des métriques

Terme	Définition
Recall (rappel)	Parmi toutes les vraies anomalies, proportion détectée. $\text{Recall} = \text{VP} / (\text{VP} + \text{FN})$
Précision	Parmi toutes les alertes émises, proportion de vraies anomalies. $\text{Précision} = \text{VP} / (\text{VP} + \text{FP})$
F1-score	Moyenne harmonique précision/rappel. $\text{F1} = 2 \times (\text{P} \times \text{R}) / (\text{P} + \text{R})$
AUC-ROC	Aire sous la courbe ROC — capacité du modèle à classer une anomalie au-dessus d'un cas normal, indépendamment du seuil. 0.5=hasard, 1.0=parfait
RUL	Remaining Useful Life — durée de vie utile restante estimée avant défaillance
Contamination	Proportion supposée d'anomalies dans les données d'entraînement (hyperparamètre des modèles non supervisés)
Stacking	Technique d'ensemble combinant les prédictions de plusieurs modèles via un méta-modèle (ici LogisticRegression)

12.2 Structure des fichiers du projet

Dossier/Fichier	Contenu
models/	Modèles ML entraînés (.pkl), scaler, PCA, seuils, métriques
data/	Jeux de données CSV (réels et synthétiques)
tests/	Suite de tests pytest (73 tests)
edge_models/ , edge_optimize.py	Outillage d'optimisation pour déploiement embarqué (Raspberry Pi)
postman/	Collections Postman pour tester l'API manuellement
reports/	Rapports générés (HTML/JSON)

12.3 Fréquences caractéristiques de défaut roulement (SKF 6205-2RS, 1450 tr/min)

Défaut	Fréquence
Bague extérieure (BPFO)	90.1 Hz
Bague intérieure (BPFI)	143.7 Hz
Bille (BSF)	59.5 Hz
Cage (FTF)	10.0 Hz

Calculées automatiquement par `signal_processing.py` selon la vitesse réelle du moteur. Normes de référence : ISO 10816-3 / ISO 20816-3.